

实验一：CDCL SAT 求解器

实验手册 2026

本实验要求同学们在给定的 SAT 求解器上修复正确性问题，并优化求解性能。实验分为两个部分：Part 1 检查求解结果的正确性，Part 2 比较求解时间。实验个人完成，允许使用 AI 编程工具，最终提交求解器源代码。

1 环境配置

请在 Linux 或 WSL 环境中完成实验。Windows 安装步骤见附录 A，macOS 安装步骤见附录 B。需要安装 GCC 12、GNU make、Python 3 和 zlib 开发包。在 Ubuntu / Debian 中，可以使用以下命令安装依赖：

```
sudo apt-get update
sudo apt-get install build-essential python3 zlib1g-dev
```

不同系统提供的 GCC 版本可能不同，可通过 `g++ --version` 检查。平台使用统一环境重新编译提交的源码；如需使用 GCC 12，可采用下方的容器方式。

下载并解压学生发布包，在工程根目录执行：

```
./configure --competition
make -j2
make part1_test
```

`configure` 生成构建配置，`make` 编译求解器，`part1_test` 运行公开正确性测试。初始版本含有正确性缺陷，因此测试可能失败。请先确认工程能够正常编译，再开始定位错误。

如果使用 Docker，可以直接使用发布包中的 Dockerfile：

```
docker build -t cdm-sat-student .
docker run --rm -u "$(id -u):$(id -g)" \
  -v "$PWD:/work" cdm-sat-student \
  sh -c './configure --competition && make -j2 && make part1_test'
```

本地机器与评测机的 CPU 和系统配置可能不同，本地用时仅用于比较自己的不同实现，最终性能以网站结果为准。

1.1 AI 编程工具配置

OpenCode 的安装与自定义模型配置见附录 C。使用编程助手打开解压后的工程目录，在工具设置中选择模型及服务商。需要 API 密钥时，请填写在工具的凭据设置中，不要写入源码。网站提交使用的个人令牌与大模型 API 密钥是两种不同的凭据。

配置完成后，确认工具能够读取工程，并在工程根目录运行编译和测试命令。若工具不能执行命令，可以手动运行后提供输出。建议使用 Git 保存各阶段的修改，便于比较和恢复。

使用 AI 工具时，应明确提供失败输入、预期结果、实际结果和允许修改的范围。修改后仍需运行测试，检查求解结果和证明是否有效。

2 SAT 求解器

2.1 输入与输出

SAT 问题要求判断一个命题公式是否存在满足赋值。本实验的输入采用合取范式 (CNF)：公式是若干子句的合取，每个子句是若干文字的析取；文字是变量或变量的否定。

例如，公式 $F = (p \vee q) \wedge (\neg p \vee q)$ 在 p 为假、 q 为真时成立，因此可满足。将它写成 DIMACS 格式，保存为 `example.cnf`：

```
c Example formula
p cnf 2 2
1 2 0
-1 2 0
```

这里用 1、2 表示 p 、 q ，负号表示否定。c 开头的行是注释；p cnf 2 2 表示 2 个变量、2 个子句；每个子句以 0 结束。仅含 0 的子句是空子句，不可能满足。

使用 `./build/coursesat example.cnf proof.out` 运行求解器。

可满足时输出 SAT 和一个满足赋值，例如 `s SATISFIABLE, v -1 2 0`；不可满足时输出 UNSAT，并将 DRAT 证明写入 `proof.out`。退出码 10 表示 SAT，20 表示 UNSAT。超时表示尚未求出结果，不能视为 UNSAT。

2.2 单元传播与决策

在部分变量已赋值的情况下，如果一个子句没有真文字，且只剩一个文字未赋值，则这个文字必须为真。这一过程称为**单元传播**。例如 p 为真时，子句 $(\neg p \vee q)$ 推出 q 为真。如果子句中的文字全部为假，就发生**冲突**，说明当前部分赋值无法扩展为满足赋值。

无法继续传播时，求解器选择一个未赋值变量并指定其取值，称为一次**决策**。每次决策建立一个新的决策层，由它引发的传播也属于这一层。第 0 层保存无需决策即可推出的赋值。赋值轨迹 (trail) 按发生顺序记录赋值；求解器同时维护决策层和传播原因。

2.3 CDCL 算法

CDCL (Conflict-Driven Clause Learning) 在冲突后分析原因，推导一个学得子句，并回跳到适当的决策层。核心流程如下：

```
重复执行：
  单元传播，直到不能继续传播或发现冲突
  如果发生冲突：
    若位于第 0 层，返回 UNSAT
    分析冲突，加入学得子句
    回跳，并利用学得子句继续传播
  否则：
    若已得到满足赋值，返回 SAT
    选择未赋值变量及其取值，建立下一决策层
```

2.4 冲突分析与子句学习

考虑以下公式片段。假设第 1 层决策 p 为真，第 2 层对与本次冲突无关的变量 z 进行决策，第 3 层决策 a 为真。其他无关子句在此省略。

$$\begin{aligned} C_1 &= (\neg a \vee b), & C_2 &= (\neg b \vee c), \\ C_3 &= (\neg b \vee d), & C_4 &= (\neg p \vee \neg c \vee \neg d). \end{aligned}$$

由 C_1 推出 b 为真，再由 C_2, C_3 推出 c, d 为真。此时 C_4 的所有文字都为假，发生冲突。下图用节点表示成立的文字，用边表示传播依赖， $\perp$ 表示冲突：

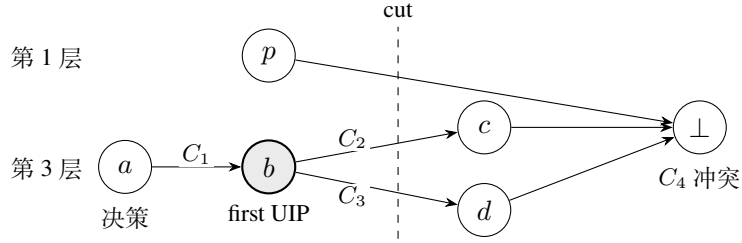


图 1: 蕴含图与 first-UIP cut。节点表示已成立的文字。

UIP (Unique Implication Point, 唯一蕴含点) 是当前决策层中的节点，当前层决策节点到冲突的每条有向路径都经过它。图中 a, b 都是 UIP；最接近冲突的 UIP 是 b ，称为 first UIP。

虚线表示一个 cut。左侧是 a, b, p ，右侧是 c, d 和冲突节点。跨过 cut 的边在左侧的起点为 b, p ，这两个条件同时成立会导致冲突，因此可以学得子句 $L = (\neg p \vee \neg b)$ 。

也可以通过消解得到 L 。将 C_4 依次与 d, c 的原因子句消解：

$$\begin{aligned} (\neg p \vee \neg c \vee \neg d), (\neg b \vee d) &\xrightarrow{\text{消去 } d} (\neg p \vee \neg c \vee \neg b), \\ (\neg p \vee \neg c \vee \neg b), (\neg b \vee c) &\xrightarrow{\text{消去 } c} (\neg p \vee \neg b). \end{aligned}$$

原公式蕴含消解得到的子句，因此加入 L 不会排除原公式的满足赋值。

2.5 非顺序回跳

L 中 b 位于第 3 层， p 位于第 1 层。撤销第 2、3 层的赋值，回到第 1 层后， p 仍为真， L 立即推出 b 为假， C_1 随后推出 a 为假。这里直接跳过了与冲突无关的第 2 层。

first-UIP 学习得到的子句仅含一个当前层文字。回跳到其余文字的最高层后，新子句成为单元子句，可以继续传播；若学得的是单元子句，则回到第 0 层。

工程实现还会使用重启、学得子句清理和公式简化等策略。重启通常撤销决策并保留学得子句；子句清理删除部分学得子句，以控制内存占用和传播开销。这些策略也是本实验可以优化的部分。

3 实验内容

3.1 Part 1: 修复正确性问题

初始代码含有正确性缺陷。你的任务是定位并修复问题，使求解器能够正确处理 CNF 输入。运行以下命令进行公开测试：

```
make part1_test
```

SAT 结果必须附带满足**原始输入公式**的赋值，UNSAT 结果必须附带有效 DRAT 证明，测试程序会独立验证两者。不能仅打印 SAT 或 UNSAT。平台还会运行**私有正确性测试 (private correctness test)**，通过本地测试不代表通过全部正确性评测。

调试时，可以从一个失败输入开始，记录实际输出和退出码，再删除无关子句、减少变量，得到较小的复现用例。变量较少时，可枚举全部赋值，独立确定正确结果。

结合赋值轨迹检查决策、传播、冲突分析和回跳过程。需要保证传播原因有效、学得子句的推导正确、回跳后的状态一致。若使用公式简化，还需保证模型恢复和证明输出正确。修复后应保留一个通过正确性测试的版本，用于后续性能比较。

3.2 Part 2: 优化求解性能

在 Part 1 的基础上修改求解策略，减少求解时间。下面列出几个可选方向，相关文件均位于 `src/`。

1. 分支选择与相位选择。决定下一步对哪个变量赋值，以及优先尝试真还是假。可从 `decide.cpp`、`score.cpp`、`rephase.cpp` 阅读。除了总用时，还可以比较决策数、冲突数和维护变量分数的开销。

2. 重启策略。调整重启时机和触发条件。可从 `restart.cpp` 阅读。频繁重启可能改善搜索方向，也可能产生较多重复传播；应同时观察重启次数、传播量和总用时。

3. 学得子句管理。调整子句保留、最小化和强化策略。可从 `reduce.cpp`、`minimize.cpp`、`vivify.cpp` 阅读。保留子句可以减少重复搜索，但会增加存储与传播成本；清理时不能随意释放仍作为传播原因的子句。

4. 公式简化。调整预处理和搜索中简化的策略、触发条件或工作预算。可从 `factor.cpp`、`congruence.cpp`、`elim.cpp`、`sweep.cpp` 阅读。简化减少的搜索开销需要超过简化本身的成本，且不能破坏模型恢复和证明。

具体改进方案由同学们通过阅读源码和实验自行确定。正式评测使用固定运行命令，最终实现必须体现在提交的源码中；仅修改本地启动脚本不会改变网站上的运行方式。

你可以选择其中一个方向深入修改，也可以组合多种策略。实验不要求提出全新的 SAT 算法，但需要保持求解正确性，并用测试比较改动的实际效果。

3.3 公开测试

学生包提供公开正确性和性能测试 (public tests)，可以在本地直接运行。**网站另有私有正确性和性能测试 (private tests)，不包含在学生包中。**

命令	内容
make part1_test	公开正确性测试，检查赋值与证明
make part2_test	公开性能测试，共 8 个输入，每题 15 秒
make public_test	依次运行上述两部分，正确性失败时停止

Part 2 会显示逐题状态、解题数和 PAR-2 总用时。命令执行结束并不表示所有题目均已解出，请检查输出中的统计结果。公开测试用于开发和比较，不并入网站排行榜。

请保持测试输入、预期答案、时限和验证程序不变。若需要增加自己的测试，另行保存输入和运行记录。

3.4 性能实验建议

每次实验尽量只改变一个主要策略，使用同一台机器、相同编译方式和相同题集比较。接近的计时结果应重复测量；若使用随机策略，还应记录随机种子。记录所有输入的结果，包括变慢或超时的输入。

例如，可以设计以下实验：

搜索过程。记录总用时、决策数、冲突数和传播量，分析时间主要消耗在哪个阶段，以及修改是否减少了相应的工作量。

子句保留。比较不同的学得子句清理策略。保留依据可以包括子句长度、活跃程度和涉及的不同决策层数量 (LBD)。观察内存使用、传播成本与解题数的变化。

公式简化。分别记录简化与搜索耗时，比较修改前后的差异。子句数量减少不一定意味着总用时下降，需要测量完整求解过程。

建议按以下形式保存实验结果，便于复现和比较：

版本	主要改动	正确性	解题数 / PAR-2
修复版	仅修复正确性	测试结果	测量结果
优化版	一项算法改动	测试结果	测量结果

用于设计和调整实现的输入与验证最终效果的输入应尽量分开。可以自行构造不同规模、不同结构的公式；小公式可通过枚举验证。变量重编号、子句重排和文字重排不改变逻辑问题，可用于检查实现是否过度依赖输入顺序，但不能代替独立题目上的验证。

使用 AI 工具辅助优化时，建议先明确实验假设和比较指标，再要求工具修改相关代码、运行测试并汇总结果。实验记录供自己分析使用，不要求随源码提交。

4 评测与提交

4.1 网站评测

网站评测包含**私有测试 (private tests)**，与学生包中的公开测试分别设置。平台统一编译提交的源码，完成公开正确性预检后，运行私有正确性测试 (private correctness test) 和私有性能测试 (private performance test)。私有测试不随学生包分发，只反馈汇总结果，不提供题目、逐题结果或运行日志。

私有性能测试共 94 个输入，每题时限 180 秒，使用单 CPU、24 GiB 内存。各提交使用相同资源限制，资源不足时排队。计时包含输入解析、预处理、搜索和证明写出；输入解压和独立验证不计入求解时间。独立验证每题最多 540 秒。

榜单按 **PAR-2 总用时** 排序，越低越好。通过验证的题目计实际求解时间；超时、崩溃、资源超限或验证超时的题目计两倍限时，即 360 秒。错误答案、缺少证明或无效证明使本次提交无效，不进入排行榜。

例如，若只有三题，前两题分别用时 12 秒、30 秒，第三题超时，则解题数为 2/3，PAR-2 为 $12 + 30 + 360 = 402$ 秒。PAR-2 是带惩罚的统计量，不是实际运行时长。

榜单显示解题数、总用时和提交时间，保留当前题集上的最佳成绩。Baseline 和助教参考实现作为对照，不占学生名次。其他学生匿名显示为 ***；登录后，本人的排名置顶并高亮，每页最多 10 项。

排行榜名次不计课程分。具体分值、截止时间和个人令牌以课程公告为准。

4.2 提交要求

请在以下实验网站提交作业、查询评测结果和查看排行榜：

<https://ipads.sjtu.edu.cn/courses/cdm/sat-lab/>

只修改并提交 `src/` 中的求解器源码。平台使用可信的构建和测试脚本，不接受通过修改测试程序或构建流程改变评测行为。

在给定工程上完成实验，不得调用外部求解器、提交预编译程序、题目数据或预先算好的答案；不得通过题名、输入哈希或查表绕过求解。使用其他代码时，应保留所需的许可证和来源说明。

完成修改后，在工程根目录执行：

```
make public_test
make handin
```

`make handin` 将 `src/` 直接下的 `.c`、`.cpp`、`.h`、`.hpp` 文件打包为 `handin.zip`。压缩包不超过 10 MiB，解压源码不超过 8 MiB、200 个文件。提交前请确认测试结果、源码版本和压缩包内容，并检查其中没有 API 密钥或个人令牌。

打开上述实验网站，输入个人令牌，在提交区上传 `handin.zip`，随后查看任务状态和最终结果。提交次数不限，每人同时最多一个待评测或评测中的任务。整套评测可能需要数小时，请预留排队和运行时间。

A Windows 环境安装

Windows 可以使用 WSL 2, 也可以使用带桌面的 Ubuntu 虚拟机。任选一种即可。已经有可用 Linux 环境的同学, 可直接按第 1 节安装编译依赖。

A.1 方案一: WSL 2

以下步骤适用于 Windows 11, 或 Windows 10 2004 (内部版本 19041) 及以上。其他版本的安装方式见微软 WSL 安装文档。

1. 安装 Ubuntu。在开始菜单搜索 PowerShell, 右键选择“以管理员身份运行”, 执行:

```
wsl --install -d Ubuntu-22.04
```

按提示完成安装并重启 Windows。首次从开始菜单打开 Ubuntu 22.04 时, 等待初始化, 创建自己的 Linux 用户名和密码。输入密码时终端不显示字符, 这是正常现象。

2. 确认使用 WSL 2。回到 PowerShell, 执行:

```
wsl -l -v
```

Ubuntu-22.04 对应的 VERSION 应为 2。若为 1, 可以执行:

```
wsl --set-version Ubuntu-22.04 2
```

以后可以从开始菜单打开 Ubuntu, 或在 PowerShell 中执行 `wsl -d Ubuntu-22.04`。进入 Ubuntu 后, 再执行本手册中的 Linux 命令。

3. 放置实验文件。在 Ubuntu 终端执行:

```
mkdir -p ~/cdm  
cd ~/cdm  
explorer.exe .
```

这会在 Windows 文件资源管理器中打开 Linux 的实验目录。将从课程网站下载的学生 ZIP 复制到这里; 随后在 Ubuntu 终端中解压。建议将工程保存在 Linux 用户目录内, 避免在 Windows 挂载目录中直接构建。

4. 编辑代码。可在 Windows 安装 VS Code 和微软发布的 WSL 扩展。在 VS Code 中连接到 WSL, 打开 Linux 内的工程文件夹; 确认集成终端运行的是 Ubuntu。也可直接使用 Linux 中的其他编辑器。

安装异常。下载一直停在 0% 时, 可按官方文档尝试 `wsl --install --web-download -d Ubuntu-22.04`。若提示虚拟化未开启, 请根据电脑厂商说明在 BIOS/UEFI 中启用硬件虚拟化, 并按提示重启。不要为了重装环境删除已有的实验文件。

A.2 方案二：VMware Workstation 虚拟机

本方案面向使用 Intel 或 AMD 处理器的 Windows 电脑。Windows ARM 设备优先使用 WSL，并确认发行版支持其架构。

1. 准备安装文件。从 VMware/Broadcom 官方入口获取 Workstation Pro，按安装向导安装。软件入口与新建虚拟机步骤见 VMware 官方说明。从 Ubuntu 22.04 官方下载页下载 `ubuntu-22.04.5-desktop-amd64.iso`。其中 amd64 同样适用于 Intel 的 64 位处理器。

2. 创建虚拟机。打开 Workstation，选择“Create a New Virtual Machine”，采用普通配置 (Typical)，选择下载的 ISO 文件作为安装介质；系统类型为 Linux / Ubuntu 64-bit。为虚拟机指定名称和保存目录。

若软件提供 Easy Install，可按提示创建账户；若选择稍后安装系统，则先创建虚拟机，再在 CD/DVD 设置中挂载 ISO，启动后进入 Ubuntu 安装界面。

3. 分配资源。本地开发可从 2 个虚拟 CPU、4 GiB 内存、40 GiB 虚拟磁盘开始；机器资源充足时可增加内存。以上是开发环境建议，不是网站评测的资源限制。请为 Windows 本身保留足够资源。网络采用默认 NAT 即可。

4. 安装 Ubuntu。启动虚拟机，选择 Install Ubuntu，依次设置语言、键盘和时区，并创建自己的用户名和密码。磁盘安装目标应为刚创建的**虚拟磁盘**；不要将宿主机的物理磁盘直接连接给虚拟机。

完成安装后重启。如果再次进入安装界面，关闭虚拟机，在 CD/DVD 设置中断开安装 ISO 后再启动。进入桌面后，可在应用菜单打开 Terminal。

5. 下载学生包。在 Ubuntu 的浏览器中登录课程网站，下载学生发布 ZIP，解压到用户目录中的实验文件夹。在该文件夹打开终端，按照第 1 节安装依赖并构建工程。

文件与账户。使用安装时自己设置的 Linux 密码执行 `sudo`。虚拟机文件中保存着完整的系统和实验代码，关闭虚拟机不会清空它们；删除虚拟机之前应先备份源码和提交包。

如果已经有其他课程安装好的 Ubuntu 虚拟机，可以复用，无需重新安装。若助教另外提供课程镜像，请使用当次公告的下载地址和账户说明。

B macOS 环境安装

macOS 使用 UTM 创建 Ubuntu 虚拟机。先在“苹果菜单 → 关于本机”中查看处理器：Apple Silicon (M 系列) 使用 ARM64 镜像，Intel Mac 使用 AMD64 镜像。镜像架构应与本机一致。

B.1 安装 UTM 与准备镜像

从 UTM 官方安装页进入官方下载入口，下载稳定版 DMG，将 UTM 拖入 Applications。已有 UTM 的同学无需重复安装。

Mac 类型	Ubuntu 22.04 安装镜像
Apple Silicon	ARM64 官方下载页，选择普通 Server install image： ubuntu-22.04.5-live-server-arm64.iso
Intel	AMD64 官方下载页，选择 Desktop image： ubuntu-22.04.5-desktop-amd64.iso

Apple Silicon 的这份镜像先安装命令行系统，下一页再介绍如何添加桌面。下载时不要选择 Raspberry Pi 预装镜像或带 largemem 后缀的镜像。

B.2 创建与安装虚拟机

1. 新建。打开 UTM，点击“+”，依次选择 Virtualize（虚拟化）和 Linux，选择从 ISO 启动，指定下载的镜像。这里使用与本机相同架构的虚拟化模式，无需选择跨架构的 Emulate。

2. 配置。可从 2 个虚拟 CPU、4 GiB 内存和 40 GiB 磁盘开始；机器资源充足时可分配更多内存。共享目录可以暂时跳过，网络保持默认设置，保存为便于识别的名称，例如 CDM-Ubuntu。菜单顺序可能随 UTM 版本变化，需核对的是镜像、架构、内存和虚拟磁盘。

3. 安装。启动虚拟机，进入 Ubuntu 安装程序。Server 安装界面使用方向键、Tab 和 Enter 操作；网络采用自动配置，不使用代理时将代理地址留空。存储选择刚创建的虚拟磁盘，按提示创建自己的用户名和密码；可跳过可选服务安装。Intel 的 Desktop 镜像按图形界面的安装向导操作。

4. 重启。安装完成后选择 Reboot。若未正常重启或又进入安装程序，停止虚拟机，在 UTM 的光驱菜单中弹出安装 ISO，再启动。此后应从虚拟磁盘进入已安装的 Ubuntu。

UTM 的 Ubuntu 安装步骤及启动故障排查见官方 Ubuntu 指南。若停在 EFI Shell，先确认镜像架构正确且安装 ISO 已挂载。

B.3 安装桌面与实验依赖

Apple Silicon 使用 Server 镜像安装后，首先看到的是命令行登录界面。输入安装时创建的用户名和密码即可使用。若需要在虚拟机中使用浏览器和图形编辑器，可添加桌面：

```
sudo apt update
sudo apt install ubuntu-desktop
sudo reboot
```

这一步下载量较大，需要保持网络连接。重启后，在图形登录界面选择自己的账户。使用 Intel Desktop 镜像或已有桌面的同学跳过这一步。只使用命令行或远程编辑器的同学也无需安装桌面。

在 Ubuntu 中打开浏览器，登录课程网站并下载学生 ZIP。将文件保存在用户目录内，再在终端完成解压和构建。不要在 macOS 的终端里执行 `apt`；依赖安装命令应在 Ubuntu 内运行。

B.4 解压、构建与检查

以下步骤同样适用于 Windows 上安装好的 WSL 或 Ubuntu 虚拟机。在 Ubuntu 终端安装解压工具和实验依赖：

```
sudo apt update
sudo apt install build-essential python3 zlib1g-dev unzip
mkdir -p ~/cdm
```

假设学生包下载到 Downloads 目录，可执行以下命令。如果实际下载位置不同，请修改 ZIP 文件路径；WSL 用户可使用上一节复制到 ~/cdm 的文件。

```
unzip ~/Downloads/sat-lab-starter-2026.zip -d ~/cdm/sat-lab
cd ~/cdm/sat-lab
./configure --competition
make -j2
make part1_test
```

检查环境。`uname -s` 应显示 Linux；`uname -m` 在 ARM64 环境显示 aarch64，在 AMD64 环境显示 x86_64。`g++ --version`、`make --version` 和 `python3 --version` 应能输出版本信息。

初始版本的正确性测试可能失败，这是实验要求修复的问题；缺少命令、编译器或依赖则属于环境问题。安装目录应包含 `configure`、`src/` 和 `test/`，不要在 ZIP 文件所在的上级目录执行构建。

完成实验后在 Ubuntu 的工程目录中执行 `make handin`，将生成的 `handin.zip` 上传到实验网站。Apple Silicon 上生成的本地可执行文件不需要提交，平台会根据源码重新编译。

C OpenCode 安装与自定义模型配置

本附录以 OpenCode 为例，介绍如何接入自定义模型服务。请先准备服务商或课程提供的三项信息：**Base URL**、**API Key** 和**模型 ID**。实验网站的提交令牌不能用作模型 API Key。

以下命令在前两节配置好的 **Ubuntu 或 WSL 终端** 中执行。OpenCode 与实验工程应位于同一个 Linux 环境中，便于直接编译和测试。macOS 用户按此流程在 Ubuntu 虚拟机内安装；Windows 用户在 WSL 或虚拟机内安装。

C.1 安装 OpenCode

根据 OpenCode 官方安装说明，使用官方脚本安装：

```
sudo apt update
sudo apt install curl ca-certificates nano
curl -fsSL https://opencode.ai/install | bash
export PATH="$HOME/.opencode/bin:$PATH"
opencode --version
```

最后一条命令应输出版本号。安装脚本会尝试更新 shell 的 PATH 配置；若新终端仍找不到命令，请在 `~/.bashrc` 中加入下面这一行，再重新打开终端：

```
export PATH="$HOME/.opencode/bin:$PATH"
```

安装需要访问 OpenCode 及其官方下载源。若下载失败，应先检查网络；已有可用 OpenCode 的同学可跳过安装，仅记录版本号。

C.2 准备配置文件与密钥文件

本教程使用用户级配置，无需修改求解器源码。需要编辑的文件如下，~ 表示当前 Linux 用户的主目录：

文件	内容
<code>~/.config/opencode/opencode.json</code>	服务地址、模型 ID，以及密钥文件的引用
<code>~/.config/opencode/cdm-api-key</code>	仅保存实际的 API Key

先创建目录和密钥文件：

```
mkdir -p ~/.config/opencode
chmod 700 ~/.config/opencode
umask 077
nano ~/.config/opencode/cdm-api-key
```

在编辑器中粘贴自己的 API Key，文件内只保留这一项内容，不添加引号、Bearer 前缀或 JSON。按 Ctrl+O、Enter 保存，按 Ctrl+X 退出，然后执行：

```
chmod 600 ~/.config/opencode/cdm-api-key
```

密钥文件存放在工程目录外，不会被 `make handin` 收集。不要把它复制到 `src/`，也不要将密钥粘贴到 agent 对话或公开截图中。

C.3 填写 Base URL 与模型 ID

本例适用于支持 **OpenAI-compatible Chat Completions** 的服务，即提供 `/chat/completions` 接口。模型还应支持工具调用，才能用于读取文件、编辑代码和执行命令；若服务提供的是其他协议，请先核对其接入说明。

在学生工程根目录执行以下命令。发布包中的 `opencode.example.json` 是配置模板；`cp -n` 在目标文件已存在时不会覆盖它。

```
cp -n opencode.example.json ~/.config/opencode/opencode.json
nano ~/.config/opencode/opencode.json
```

若已有配置，请将下面的 `provider` 合并到原文件中，并检查默认模型字段。新配置的完整内容如下：

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "course/REPLACE_MODEL_ID",
  "provider": {
    "course": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "Course Model Provider",
      "options": {
        "baseUrl": "https://api.example.invalid/v1",
        "apiKey": "{file:~/.config/opencode/cdm-api-key}"
      },
      "models": {
        "REPLACE_MODEL_ID": {
          "name": "Course Model"
        }
      }
    }
  }
}
```

需要修改两处信息：

Base URL: 将 `https://api.example.invalid/v1` 替换为提供给你的实际 API 根地址。这是占位地址，不能用于连接。许多服务的根地址以 `/v1` 结尾，但应以服务商说明为准；不要填模型平台的网页地址，也不要末尾添加 `/chat/completions`。

模型 ID: 将两处 `REPLACE_MODEL_ID` 都替换为服务端实际接受的模型 ID，保留大小写和完整名称。顶层 `model` 使用“provider ID / 模型 ID”格式；这里 provider ID 是自定义的 `course`。`name` 只是界面显示名称。

`apiKey` 中的 `{file:...}` 会读取上一页创建的密钥文件，保留这个引用即可。保存 JSON 时使用英文双引号，不要在最后一个字段后添加逗号。字段格式依据自定义 `Provider` 文档，文件引用依据配置文档。

示例：交大 DeepSeek 接口

交大 DeepSeek 接口的 Base URL 为 `https://models.sjtu.edu.cn/api/v1`，模型调用名为 `deepseek-chat`（常规模式）和 `deepseek-reasoner`（思考模式）。请使用该平台提供的 API Key；可用模型与额度以平台说明为准。

本例沿用 C.2 创建的 `cdm-api-key` 密钥文件。在 `~/.config/opencode/opencode.json` 中填写以下配置；已有配置时，合并 `deepseek provider` 并调整默认 `model`，保留其他服务商。

```
{
  "$schema": "https://opencode.ai/config.json",
  "model": "deepseek/deepseek-chat",
  "provider": {
    "deepseek": {
      "options": {
        "baseUrl": "https://models.sjtu.edu.cn/api/v1",
        "apiKey": "{file:~/.config/opencode/cdm-api-key}"
      },
      "whitelist": ["deepseek-chat", "deepseek-reasoner"],
      "models": {
        "deepseek-chat": {
          "name": "DeepSeek Chat (SJTU)",
          "reasoning": false
        },
        "deepseek-reasoner": {
          "name": "DeepSeek Reasoner (SJTU)",
          "reasoning": true
        }
      }
    }
  }
}
```

保存后在终端执行：

```
opencode models deepseek
```

模型列表应包含：

- `deepseek/deepseek-chat`：常规模式。
- `deepseek/deepseek-reasoner`：思考模式。

重启 OpenCode，输入 `/models` 选择模型，再按 C.4 验证实际连接。本例的 provider ID 为 `deepseek`；C.4 的列出模型命令请改用上方命令。

本例使用 C.2 的密钥文件引用，密钥保存在工程目录之外。模型选项与文件引用格式见 OpenCode Provider 文档及配置文档。

C.4 检查配置并连接模型

先在 Linux 终端检查 JSON 语法，并列出该 provider 下的模型：

```
python3 -m json.tool ~/.config/opencode/opencode.json >/dev/null
opencode models course
```

第一条命令成功时不输出内容；第二条应列出 `course/` 实际模型 ID。列出模型仅说明 OpenCode 识别了配置，并不代表 API 已连接成功。命令说明见 OpenCode CLI 文档。

进入实验工程目录并启动 OpenCode：

```
cd ~/cdm/sat-lab
opencode
```

在 OpenCode 界面中输入 `/models`，选择刚配置的模型。随后发送：“只回复连接成功，不读取或修改文件。”收到模型回复后，再测试读取 README、说明构建命令等操作。后者用于检查工具调用能力，不能仅凭普通对话成功就认为编程功能可用。

如需更换模型，修改 JSON 中的模型 ID；如需更换服务地址，修改 `baseURL`；如需更换密钥，编辑 `cdm-api-key`。保存后退出并重新启动 OpenCode。本教程使用文件引用，不需要再执行 `/connect`；若另行使用 `/connect`，应避免与这里的凭据配置混用。

C.5 常见问题

现象	检查内容
找不到 opencode	是否在安装它的 Linux 环境中运行；PATH 是否包含安装目录
JSON 解析失败	英文引号、括号配对、逗号，以及是否把说明文字复制进文件
模型没有出现在列表中	两处模型 ID 是否一致；工程内的 <code>opencode.json</code> 是否覆盖用户配置
401 / 403	密钥是否正确、是否有效，账号是否获准使用该服务或模型
404 / model not found	Base URL 的路径和模型 ID；是否误加接口后缀或选错 API 协议
网络连接失败	虚拟机网络、代理、DNS，以及服务地址是否可从当前环境访问
能聊天但不能操作文件	模型的工具调用支持、服务兼容性，以及当前权限提示

用户配置与工程配置可能合并，工程中的同名设置通常覆盖用户级设置，详见配置文件说明。处理连接问题时，记录版本号、错误状态和已脱敏的配置，不要展示密钥文件内容。

连接完成后，再要求 agent 阅读实验手册与源码、运行公开测试并分析结果。实验提交仍然只包含求解器源码；OpenCode 配置、密钥文件和对话记录均不需要上传。